PES-VCS Project Report

Overview

This repository contains a working implementation of PES-VCS, a small local version control system built in C. It stores file contents as hashed objects, builds trees from staged files, records commit history, and updates branch references through a HEAD pointer.

What Changed

The implementation is split across four core source files:

- `object.c`: stores and reads content-addressed objects with integrity checks
- `tree.c`: builds tree objects from the staged index
- `index.c`: loads, saves, and updates the staging area
- `commit.c`: creates commits and updates repository history

The result follows a simplified Git workflow:

1. `pes add` stores file data in the object database and updates the index.
2. `pes commit` builds a tree from the index and writes a commit object.
3. `pes log` walks the commit chain through the branch reference.

Build And Test

```
make
make test_objects
make test_tree
make test-integration
```

The implementation was verified with the object tests, tree tests, and the full integration sequence.

Screenshots

Phase 1

```
└─❮ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
┌─ ~/projects/os-u4-orange 〉 gitmain ⇡5 ?1
└─❮ find .pes/objects -type f
```

Phase 2

```
~/projects/os-u4-orange > gitmain ⇡5 ?1
❮ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
~/projects/os-u4-orange > gitmain ⇡5 ?1
❮ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.

~/projects/os-u4-orange > gitmain ⇡5 ?1
❮ find .pes/objects -type f

~/projects/os-u4-orange > gitmain ⇡5 ?1
❮ xxd .pes/objects/XX/YYY | head -20
xxd: .pes/objects/XX/YYY: No such file or directory
```

Phase 3

```
~/projects/os-u4-orange  gitmain ⇡5 ?2
❮ ./pes init
Initialized empty PES repository in .pes/

~/projects/os-u4-orange  gitmain ⇡5 ?2
❮ echo "hello" > file1.txt

~/projects/os-u4-orange  gitmain ⇡5 ?2
❮ echo "hello2" > file2.txt

~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ ./pes add file1.txt file2.txt

~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_objects
  untracked:  test_tree
```

```
~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776325413 6 file1.txt
100644 bb7453659ced8100324e1377f9b4388f5740236bb8be1ffb27d2a74940422963 1776325425 7 file2.txt
```

Phase 4

```
~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ ./pes log
commit 6d757a18a0aa56d3faf732dc2c7f854276ddc5de6fd393ccfb0df3aa438a138b
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776325299

    init for screenshot

commit 1ec7a2ce8c65c51c5a62201765ce2121e622d6f51831101099c2d9ae67eec90d
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776325290

    init for screenshot screenshotA
```

```
~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/09/b576c2eb51b78fcf9b8493a0909522a75f8ee86df0e926312deeb5fbe7dfb1
.pes/objects/1e/c7a2ce8c65c51c5a62201765ce2121e622d6f51831101099c2d9ae67eec90d
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/6d/757a18a0aa56d3faf732dc2c7f854276ddc5de6fd393ccfb0df3aa438a138b
.pes/objects/bb/7453659ced8100324e1377f9b4388f5740236bb8be1ffb27d2a74940422963
.pes/refs/heads/main
```

```
~/projects/os-u4-orange  gitmain ⇡5 ?3
❮ cat .pes/refs/heads/main
6d757a18a0aa56d3faf732dc2c7f854276ddc5de6fd393ccfb0df3aa438a138b
```

Final

```
export PES_AUTHOR="Andey Hemanth <PES2UG24CS061>" && make clean && make test
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
=== Running Phase 1 tests ===
./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc1
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.

=== Running Phase 2 tests ===
./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: f135a34e0f0e... Initial commit

Log after first commit:
commit f135a34e0f0ece5cbe455167bf6f69e1bcdb5cffe7007bbfcba1536e1d778300
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776323918

    Initial commit!


--- Second Commit ---
Committed: f752a8d63c3f... Update file.txt

--- Third Commit ---
Committed: 9c0acf93113d... Add farewell

--- Full History ---
commit 9c0acf93113daa1546da4652ea9d828b52a2d4da6cacc0f8d6711e76e86eaac2
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776323918

    Add farewell

commit f752a8d63c3f1f3fc73dad27ea553dfca591738044633c4d6a3148d0887127d3
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776323918

    Update file.txt

commit f135a34e0f0ece5cbe455167bf6f69e1bcdb5cffe7007bbfcba1536e1d778300
Author: Andey Hemanth <PES2UG24CS061>
Date:   1776323918

    Initial commit!


--- Reference Chain ---
HEAD:
ref: refs/heads/main
```

```
refs/heads/main:
9c0acf93113daa1546da4652ea9d828b52a2d4da6cacc0f8d6711e76e86eaac2

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/9c/0acf93113daa1546da4652ea9d828b52a2d4da6cacc0f8d6711e76e86eaac2
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f1/35a34e0f0ece5cbe455167bf6f69e1bcdb5cffe7007bbfcba1536e1d778300
.pes/objects/f7/52a8d63c3f1f3fc73dad27ea553dfca5917380446 33c4d6a3148d0887127d3

=== All integration tests completed ===
```

Notes

The screenshots show the staged repository workflow, object storage layout, tree serialization output, commit history, reference chain, and the final integration test run.

Analysis Answers

Q5.1 Branching And Checkout

To implement pes checkout <branch>, these updates are needed:

- Validate that .pes/refs/heads/<branch> exists.
- Update .pes/HEAD to point to that branch reference.
- Read the target commit hash from .pes/refs/heads/<branch>.
- Load the target commit tree and rewrite the working directory to match it.
- Update .pes/index to match the checked-out tree.

The hard part is the working directory update. Files can be added, removed, modified, or conflict with local edits. The command must avoid destroying uncommitted work.

Q5.2 Dirty Working Directory Detection

A safe check can be done in three comparisons:

1. Compare current working files with .pes/index using metadata and content hash when needed.
2. Compare current branch tree with target branch tree to find paths that differ between branches.
3. If any path is both locally modified and branch-different, abort checkout.

This blocks destructive overwrites and keeps behavior consistent with Git-like safety rules.

Q5.3 Detached HEAD

Detached HEAD means HEAD points directly to a commit hash, not a branch file. New commits still work, but they are not attached to a named branch pointer. If the user moves away, those commits can become hard to find.

Recovery options:

- Create a new branch at that commit hash.
- Move an existing branch ref to that commit hash.

As long as the hash is known from log or object walk, the commits can be preserved.

Q6.1 Garbage Collection Algorithm

A mark-and-sweep approach works:

1. Start from all branch heads in .pes/refs/heads.
2. Walk each reachable commit.
3. For each commit, mark its tree and parent commit.
4. For each tree, recursively mark child trees and blobs.
5. After marking, scan .pes/objects and delete unmarked objects.

Use a hash set for marked object IDs for fast O(1) membership checks.

For 100,000 commits and 50 branches, commit traversal is about 100,000 unique commits in a shared history case, plus all unique reachable trees and blobs. Exact total depends on file churn, but the visited object count can be several times the commit count.

Q6.2 GC Race Condition With Commit

GC is unsafe during commit without coordination. A race can happen like this:

- Commit process writes new blob/tree objects first.
- Before branch ref is updated, GC scans refs and does not see those new objects as reachable.
- GC deletes them.
- Commit then writes a commit object pointing to missing data.

Git avoids this using lock files, atomic ref updates, and conservative GC rules that avoid deleting recent loose objects immediately. In practice, write operations and GC are coordinated so objects are not collected between object creation and ref publication.